2026 Testing Policy

Uni Textbook Marketplace

For **Demo 2**

Presented by:

NEXUSDEV

Testing Policy

Project: Uni Textbook Marketplace
Team: NexusDev
Client: Agile Bridge
University: University of Pretoria - COS 301 Software Engineering 2026
Document version: 1.0 - Demo 2
Last updated: 30 July 2026

The Team

Team Member	Student number
1. Tiego Mokwena*	22496336
2. Gift Mohuba	23545527
3. Neo Bosoga	23591732
4. Josh Kretschmer	22883925

Contents

1 Introduction **3**

2 Testing Objectives **4**

3 Testing Types **5**

3.1 Unit Testing 5

3.2 Integration Testing 5

3.3 End-to-End (E2E) Testing 5

3.4 Manual and Exploratory Testing 5

3.5 Accessibility Testing 6

4 Testing Tools and Environments **7**

4.1 Environments 7

5 Coverage Targets **8**

6 Defect Management Process **9**

6.1 Workflow 9

6.2 Severity Levels 9

7 Acceptance Criteria **11**

1. Introduction

This document lays out how we approach testing on the Uni Textbook Marketplace project: what we test, how we test it, what tools we use, how we handle bugs when we find them, and what has to be true before a feature is considered done. The point of writing this down isn't to look impressive, it's so all four of us are actually testing things the same way instead of everyone deciding their own bar for "good enough" on their own features.

Like the Coding Standards document, most of what's in here isn't just policy on paper, it's enforced by our CI pipeline. If a pull request doesn't have passing tests, it doesn't merge, full stop.

2. Testing Objectives

- Catch regressions before they reach `develop` or `main`, not after a teammate (or a marker) finds them.
- Verify that each use case actually does what its functional requirements say it should, not just that the code runs without throwing an error.
- Work toward the Maintainability quality requirement from the SRS, an 80% automated test coverage target, so the codebase stays safe to change as the project grows.
- Make sure critical flows (auth, listing creation, moderation, messaging) keep working even as unrelated parts of the system change underneath them.
- Keep the system accessible, WCAG 2.2 AA as the baseline, checked with real Lighthouse audits rather than just assumed.

3. Testing Types

3.1 Unit Testing

Backend unit tests use Jest and test a single service or controller in isolation, with its dependencies (repositories, other services) mocked out. These are meant to be fast and to test logic, not infrastructure, things like validation rules, price calculations, or status transitions on a listing.

Frontend unit tests also use Jest, together with React Testing Library, and focus on individual components: does a form show a validation error when it should, does a button call the right handler, that kind of thing.

3.2 Integration Testing

Backend integration tests also run through Jest, but instead of mocking the database, they run against a real, temporary Postgres container that spins up inside CI. This matters because a lot of the bugs we've actually hit (see the Coding Standards document's migration section) only show up against a real database, a mocked repository would have happily let all of them through. Integration tests cover things like a controller endpoint actually persisting a row correctly, guards correctly blocking an unauthorised role, and TypeORM relations resolving the way we expect.

3.3 End-to-End (E2E) Testing

Backend E2E tests use Supertest against a running instance of the full NestJS application, hitting real HTTP routes the way a client would, covering full request/response cycles including auth, guards, and database writes.

> **Known gap, being upfront about it:** Cypress end-to-end testing for the frontend is planned but not yet implemented as of Demo 2. Frontend coverage right now is Jest unit/component tests only. This is tracked as a follow-up item and is not being hidden or glossed over, we'd rather state it plainly here than have it discovered during marking.

3.4 Manual and Exploratory Testing

Before approving a pull request, the reviewer doesn't just read the diff, they pull the branch (or check the relevant staging deployment) and actually click through the changed flow themselves. This has caught things automated tests missed, particularly UI states that only show up with specific data (empty states, long text overflow, etc.).

3.5 Accessibility Testing

Each major page is audited with Google Lighthouse against WCAG 2.2 AA as the baseline target. Current scores, taken from our own Lighthouse runs:

Page	Accessibility Score
Landing Page	94 / 100
Listings Browse Page	89 / 100
Create Listing Page	94 / 100
In-App Messaging	95 / 100
Wishlist Page	96 / 100
Audit Log Page	85 / 100

The two lower scorers, Listings Browse and Audit Log, are both pages with a lot of interactive and tabular content on screen at once (filters, grids, data tables), which naturally gives Lighthouse more contrast and labelling checks to fail on. These are tracked as follow-up items rather than being treated as resolved.

4. Testing Tools and Environments

Tool	Used for	Notes
Jest	Backend unit + integration tests, frontend unit tests	Runs in `ci.yml` on every push and PR
Supertest	Backend E2E tests	Hits real routes on a running NestJS instance
Postgres (containerised)	Integration/E2E test database	Ephemeral, spun up fresh inside CI, not the staging or production database
React Testing Library	Frontend component tests	Used alongside Jest
Cypress	Frontend E2E tests	Planned, not yet implemented
Codecov	Coverage tracking	Separate flags for backend and frontend
SonarCloud	Static analysis, code smells, security hotspots	Runs on every push/PR to `main` and `develop`
Lighthouse	Accessibility auditing	Run manually against deployed pages
GitHub Actions	CI/CD orchestration	Runs all of the above and gates merges on the result

4.1 Environments

- **Local:** Docker Compose, run against a local Postgres container, used for day-to-day development before anything is pushed.
- **CI (ephemeral):** A fresh Postgres container created for the lifetime of the pipeline run only, used for integration and E2E tests. Nothing here ever touches staging or production data.
- **Staging:** The `develop` branch auto-deploys here. This is where manual/exploratory testing of a feature happens before it's considered safe to promote to production.
- **Production:** The `main` branch. Nothing merges here without having already passed through staging first.

5. Coverage Targets

The SRS sets an 80% automated test coverage target under the Maintainability quality requirement. We track actual coverage through Codecov, with separate badges/flags for backend and frontend so one side doing well doesn't hide the other side lagging behind.

> **Being honest about where we are:** 80% is the target we're working toward, not a hard gate that currently blocks merges on its own. Right now, merge is gated on the test suite passing and SonarCloud's Quality Gate, not on a specific coverage percentage threshold. Tightening this into an enforced minimum is a planned follow-up rather than something already in place.

6. Defect Management Process

We track bugs as GitHub Issues on our project board (`github.com/orgs/COS301-SE-2026/projects/64`) the same board we use for feature work, rather than keeping a separate bug list somewhere else.

6.1 Workflow

1. **Report:** Anyone who finds a bug (a team member, a mentor, or a marker during a demo) opens a GitHub Issue with the `bug` label, describing the steps to reproduce it and the expected vs. actual behaviour.
2. **Triage:** The issue is assigned a severity (see below) and assigned to whoever owns the relevant feature area.
3. **Fix:** A fix branch is created off `develop`, following the same `fix/short-description` naming convention as the Coding Standards document, and a regression test is written that would have caught the bug in the first place, not just a fix without a test behind it.
4. **Review and merge:** The fix goes through the normal PR process, one reviewer, `ci-success` passing, before it merges into `develop`.
5. **Verify:** The reporter (or the fix's author, if the reporter isn't available) confirms the fix actually works on staging before the issue is closed.

6.2 Severity Levels

Severity	Meaning	Expectation
Critical	System is down, or a core flow (auth, listing creation, moderation) is broken for everyone	Fixed immediately, before other work continues
High	A feature is broken or badly degraded, but there's a workaround or it's not core	Fixed within the current sprint
Medium	A feature works but behaves incorrectly in an edge case	Fixed when the owning team member has capacity

Severity	Meaning	Expectation
Low	Cosmetic issue, typo, minor styling inconsistency	Fixed opportunistically, batched with other small fixes

7. Acceptance Criteria

Before a use case is considered "done" and demo-ready, all of the following need to be true:

- Unit tests exist for the new service/controller logic, and integration or E2E tests exist for the new endpoint(s), using a real database in CI rather than a mock.
- The feature has been manually tested against the acceptance criteria written in its GitHub Issue, not just against the happy path.
- The pull request has at least one approving review from a teammate other than the author.
- The `ci-success` check is green: lint, tests, and SonarCloud's Quality Gate all pass.
- The feature has been checked on the staging deployment (not just locally) before `develop` is merged into `main`.
- Where the feature has a UI, it's been checked against the Brand Style Guide and meets WCAG 2.2 AA, no unlabelled inputs, no insufficient colour contrast, keyboard navigable.

Uni Textbook Marketplace

In collaboration **with:**

agilebridge
CONSIDERED
SOFTWARE SOLUTIONS

NEXUSDEV

This document is version-controlled in the GitHub repository under `/docs/Demo 2/Testing Policy.pdf`